Packages:

- A Package can be defined as a grouping of related types(classes, interfaces)
- A package represents a directory that contains related group of classes and interfaces.
- Packages are used in Java in order to prevent naming conflicts.
- There are two types of packages in Java.
    1. Pre-defined Packages(built-in)
    2. User defined packages

Pre-defined Packages:

Package Name	Description
java.lang	Contains language support classes (for e.g classes which defines primitive data types, math operations, etc.). This package is automatically imported.
java.io	Contains classes for supporting input / output operations.
java.util	Contains utility classes which implement data structures like Linked List, Hash Table, Dictionary, etc and support for Date / Time operations. This package is also called as **Collections.**
java.applet	Contains classes for creating Applets.
java.awt	Contains classes for implementing the components of graphical user interface (like buttons, menus, etc.).
java.net	Contains classes for supporting networking operations.
javax.swing	This package helps to develop GUI Applications. The 'x' in javax represents that it is an extended package which means it is a package developed from another package by adding new features to it. In fact, javax.swing is an extended package of java.awt.
java.sql	This package helps to connect to databases like Oracle/Sybase/Microsoft Access to perform different operations.

Defining a Package(User defined):

To create a package is quite easy: simply include a **package** command as the first statement in a Java source file. Any classes declared within that file will belong to the specified package. The **package** statement defines a name space in which classes are stored. If you omit the **package** statement, the class names are put into the default package, which has no name.

This is the general form of the **package** statement:

package *pkg*;

Here, *pkg* is the name of the package.

For example, the following statement creates a package called **MyPackage**:

package MyPackage;

Java uses file system directories to store packages. For example, the **.class** files for any classes you declare to be part of **MyPackage** must be stored in a directory called **MyPackage**. Remember that case is significant, and the directory name must match the package name exactly. More than one file can include the same **package** statement.

Most real-world packages are spread across many files. You can create a hierarchy of packages. To do so, simply separate each package name from the one above it by use of a period. The general form of a multileveled package statement is shown here:

package *pkg1*[.*pkg2*[.*pkg3*]];

A package hierarchy must be reflected in the file system of your Java development system. For example, a package declared as

package java.awt.image;

Example: Package demonstration

```
package pack;
public class Addition
{
        int x,y;
        public Addition(int a, int b)
        {
                x=a;
                y=b;
        }
        public void sum()
        {
                System.out.println("Sum :"+(x+y));
        }
}
```

Step 1: Save the above file with Addition.java

```
package pack;
public class Subtraction
{
```

```
        int x,y;
        public Subtraction(int a, int b)
        {
                x=a;
                y=b;
        }
        public void difference()
        {
                System.out.println("Difference :"+(x-y));
        }
}
```

Step 2: Save the above file with Subtraction.java

Step 3: Compilation

To compile the java files use the following commands

javac -d directory_path name_of_the_java file

Javac –d . name_of_the_java file

Note: -d is a switching option creates a new directory with package name. Directory path represents in which location you want to create package and . (dot) represents current working directory.

```
F:\Courses\JAVA26-27\Unit4\Packages>javac -d . Addition.java

F:\Courses\JAVA26-27\Unit4\Packages>javac -d . Subtraction.java
```



Step 4: Access package from another package

There are three ways to use package in another package:

1. **With fully qualified name.**

```
class UseofPack
{
        public static void main(String arg[])
        {
                pack.Addition a=new  pack.Addition(10,15);
                a.sum();
                pack.Subtraction s=new pack.Subtraction(20,15);
```

```
                s.difference();
        }
}
```

2. **import package.classname;**

```
import pack.Addition;
import pack.Subtraction;
class UseofPack
{
        public static void main(String arg[])
        {
                 Addition a=new  Addition(10,15);
                a.sum();
                Subtraction s=new Subtraction(20,15);
                s.difference();
        }
}
```

3. **import package.*;**

```
import pack.*;
class UseofPack
{
        public static void main(String arg[])
        {
                 Addition a=new  Addition(10,15);
                a.sum();
                Subtraction s=new Subtraction(20,15);
                s.difference();
        }
}
```

Note: Don't place Addition.java, Subtraction.java files parallel to the pack directory. If you place JVM searches for the class files in the current working directory not in the pack directory.

How Java Finds a Package (Class Path Resolution):

When a program uses a package, the compiler and JVM must locate the package's .class files. There are **three ways** to tell Java where the package is.

1. Current Working Directory (default): By default, Java starts searching from the current working directory. If the package directory is a subdirectory of the current directory, it is found automatically.

```
F:\Packages\
├── UseofPack.java
└── pack\
    ├── Addition.class
    └── Subtraction.class
```

```
F:\...\Packages > javac UseofPack.java
F:\...\Packages > java UseofPack
```

2. CLASSPATH Environment Variable: Set CLASSPATH to the directory that contains the package directory (the *parent* of pack).

Example: UseofPack.java is in the E drive, and package pack is in the F drive.

```
F:\.....\Packages\
└── pack\
    ├── Addition.class
    └── Subtraction.class

E:>
└── UseofPack.java
```

```
E:> set CLASSPATH=F:\.....\packages;.;
E: > javac UseofPack.java
E: > java UseofPack
```

```
E:\>set CLASSPATH=F:\Courses\JAVA26-27\Unit4\Packages;.;
```

3. The -cp (or -classpath) Option

Pass the path directly with the java and javac commands. This applies **only to that command** and does not change any system settings.

```
E:\>javac -cp "F:\Courses\JAVA26-27\Unit4\Packages;.;" UseofPack.java

E:\>java -cp "F:\Courses\JAVA26-27\Unit4\Packages;.;" UseofPack
Sum :25
Difference :5
```

Here too, include . when the class you are running is in the current directory.